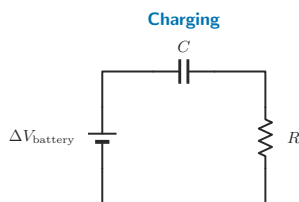


Chapter 6

Resistors and Capacitors

In this lab, we will not build a new instrument. We will use an instrument we built in a previous lab (or at least only a new version of that instrument adjusted for today's resistance values). You will notice a pattern in what we do from now on in PH250. We will build an instrument and then test a model with that instrument. The instrument must be designed so that it can take the data needed to test the model. In today's lab, we will test the model of how capacitors work in a circuit. If your PH220 class is moving along nicely, this model will be familiar.

6.1 The Model to Test



Let's start by thinking of hooking up a capacitor and a resistor in series with a battery. The capacitor will become charged. The voltage across the capacitor as a function of time will be given by

$$\Delta V_C(t) = \Delta V_{\text{max}} \left(1 - e^{-\frac{t}{\tau}}\right) \quad \text{Charging}$$

where

$$\tau = RC \tag{6.1}$$

is the product of the resistance, R , and the capacitance, C . The current in the circuit as a function of time will be given by

$$I(t) = I_{\max} e^{-\frac{t}{\tau}} \quad \text{Charging}$$

while we charge up the capacitor. The quantity

$$\tau = RC$$

is called the time constant.

We should review what a time constant is. Think of a particular case, say,

$$\begin{aligned} \Delta V_{\text{battery}} &= 1.5\text{V} \\ R &= 2\Omega \\ C &= 10\text{F} \end{aligned}$$

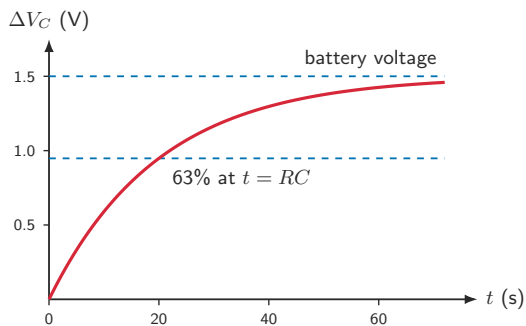
then

$$V_C(t) = (1.5\text{V}) \left(1 - e^{-\frac{t}{(2\Omega)(10\text{F})}} \right)$$

and

$$\begin{aligned} \tau &= (2\Omega)(10\text{F}) \\ &= 20.0\text{s} \end{aligned}$$

We can plot this



Notice, that by about $t = 70\text{s}$ we essentially have $\Delta V_C = \Delta V_{\text{battery}}$. But up to that point, the voltage across the capacitor changes in a very non-linear way. The part of the equation that looks like

$$\left(1 - e^{-\frac{t}{RC}} \right)$$

is interesting. What is e^0 ?

$$e^0 = 1$$

So at $t = 0$ we do have $\Delta V_C = 0$ on the capacitor because

$$\left(1 - e^{-\frac{t}{RC}}\right) = (1 - 1)$$

For any positive time, $e^{-\frac{t}{RC}}$ will be less than 1. For large positive times $\frac{t}{RC}$ gets to be a big number. So $e^{-\frac{t}{RC}}$ gets very small. Then $\left(1 - e^{-\frac{t}{RC}}\right)$ gets very close to 1. That means that

$$\lim_{t \rightarrow \infty} \Delta V_C = \lim_{t \rightarrow \infty} \Delta V_{battery} \left(1 - e^{-\frac{t}{RC}}\right) = \Delta V_{battery} (1) = \Delta V_{battery}$$

just as we saw in the graph and as we know it must.

But what if $t = \tau = RC$? Then

$$\begin{aligned} \Delta V_C &= \Delta V_{battery} \left(1 - e^{-\frac{RC}{RC}}\right) = \\ &= \Delta V_{battery} (1 - e^{-1}) \\ &= 0.63212 \Delta V_{battery} \\ &\approx 63\% \Delta V_{battery} \end{aligned}$$

The time τ is the time it takes for the capacitor to be 63% charged.

The quantity τ is called the *time constant* because it tells us something about how long it takes for ΔV_C to go from 0 to get to $\Delta V_{battery}$. The “t-looking-thing” is a Greek letter “t.” It is pronounced “tau.” This quantity will be useful in planning your experiment.

Notice what we have done. We have used our model to form an equation, and we have used part of that equation to understand how much time it will take to perform a test (experiment) of the model. This is typical, get an idea of how to make the measurement from the model we are testing.

But! you say, I don’t really remember where all of these equations came from. Or maybe your PH220 class hasn’t gotten to allowing current to flow yet so you have not done this. If any of this is mysterious, please read the section of our PH220 book that covers RC circuits. But if it is vaguely familiar or seems to make sense, really we can test our model of how capacitors work just knowing a little about capacitors and the equations that came from the model.

6.2 The Instrument

To test our capacitor model we need to measure the voltage across the capacitor as a function of time. We could also measure the current in the circuit as a function of time. One of these is sufficient to test the model. I am going to describe measuring the voltage across the capacitor as a function of time. But you know from a previous lab how we might add current as a function of time.

We need a device that measures voltage and how it changes as a function of time. But that is just what our Arduino’s do! We already know how to build


```

// so make a place to store a time value
unsigned long time;

// make some integer variables that identify the
// analog input pins we will use:
int AI0 = 0;
int AI1 = 1;

// you also need a place to put the analog to
// digital converter values
// from the Arduino
int ADC0 = 0;
int ADC1 = 0;

// Remember we will have to convert from Analog to
// digital converter(ADC) units to volts.
// We need our delta_V_min just like we did in lab 3
float delta_v_min=0.0049;    // volts per A2D unit

// We need a place to put the calculated voltage
float voltage = 0.0;

////////////////////////////////////
void setup() {
    // put your setup code here, to run once:
    //Initiate Serial Communication
    Serial.begin(9600);    //9600 baud rate
}

////////////////////////////////////
void loop() {
    // Read in the voltages in A2D units form the
    // serial port
    ADC0 = analogRead(AI0);
    ADC1 = analogRead(AI1);

    // Convert the voltage across the
    // test resistor to voltage
    // units using delta_v_min
    voltage = (ADC1-ADC0) * delta_v_min;

    // output the time since we started and the voltage
    Serial.print("_time_in_sec,_");
    // this next line uses millis() which gives time in
    // ms since we started
    time = millis();

```

```

        // convert to seconds and print.
        Serial.print(time/1000.0, 6);
        Serial.print(",_voltage_across_C,_");
        Serial.println(voltage,6);
    }
    //////////////////////////////////////////
    //////////////////////////////////////////

```

This is just a voltmeter, but one with two A2D pins and a ground. This sketch also gives us time using the `millis()` function. This function gives the number of milliseconds since our experiment began. We can use our python code from a previous lab to save the data into a file. I modified the previous code just a bit, so here is an updated version.

[Download here](#)

```

#-----
#-----
# Python Code to read a stream of data from the serial port
# and save it to a file
#-----
# The idea is to read a series of voltages from an Arduino
# connected to the serial port where the Arduino is being
# used as the Analog to Digital converter. Both the voltage
# and the time the voltage was taken are sent to the serial port.
#
# We will use two libraries, serial and time
# The serial library is used to read the serial port
# The time library is used to space out our data collection by
# adding a delay in between data points. The amount of time
# to wait in between data points is called "timeBetween."
#
# We may have to install the serial library. If you have the
# Anaconda Python for Windows, you can open an Anaconda
# window and use the command 'conda install pyserial'
# This must be done before the code can run.
#
# Debugging issues: The Anaconda Python distribution tends to
# hang on to the serial port even if the program does not run.
# If this happens, try sending the python command ser.close()
# at the command prompt. If this doesn't work, You may have to
# restart Python.
# In windows, closing (after saving) the IDE and reopening it
# might be enough.
#-----
# import libraries
import serial
import time

# define variables for the delay time we wait between data points
timeBetween=4 #seconds

# define the number of data points to collect
N=40

```

```

#the next line opens a file and creates a pointer or handle for that
# file
# to use as a reference. You have to use double slashes in the path.
# The pointer, "dataFile" takes the place of all the path and file
# name so it is easier to use in the code below
dataFile=open("C:\\Users\\rtlines\\Documents\\RCdata.csv","w")

#the next line opens the serial port for communication
ser=serial.Serial('COM3', baudrate = 9600, timeout=1)
#the next line clears out the serial port so we get clean data.
ser.flushOutput()

#there will be a delay before the first data point comes from the
# serial port, warn the user so they don't worry.
print('getting started...')

# set our index to zero
i=0

# Now for N points, collect data and write it to a file
while (i<N): #Begin data collection loop
    #We will take data every "timeBetween" seconds. We need to know
    # when we start waiting so we can tell if it is time to collect
    # data yet. Use the time.time() to get the current time in
    # seconds
    # since Jan 1,1970. Yes that is a weird way to measure time, but
    # computers do it this way.
    waitStart=time.time()

    #Data comes to the serial port fast. We will continually collect
    # the data as fast as it comes, but only save it every timeBetween
    # seconds. The next while loop keeps us reading in data, but only
    # when the current time - waitStart >= timeBetween will we use
    # the data.
    while (time.time()-waitStart<timeBetween): #Begin Data read loop
        # Get data from the serial port
        # it should have a time and a voltage
        arduinoData=ser.readline().decode('ascii')
        # end of the Data read loop

    # the next line just prints the voltage point on the console so the
    # user
    # feels like something is happening.
    print(arduinoData)
    # This next line writes the time since we started and the Arduino
    # value from the serial port into one string. That seems simple
    # enough. But there is a historical issue that makes this harder.
    # Windows puts in two characters to say we have reached the end
    # of a line. They are no printing characters so lets represent
    # as as <CR> and <LF>. Linux just uses one <LF> and some macs
    # use <LF> and some <CR>. Sigh. If we arn't careful, we can use
    # the wrong one and get an extra blank line on some systems.
    # The next line gives the option to add in one or two of these
    # non-printing characters. Usually just printing what we read
    # in is fine. You might need to add a "new line" version of the
    # non printing new line characters by adding a \n to the end of

```

```

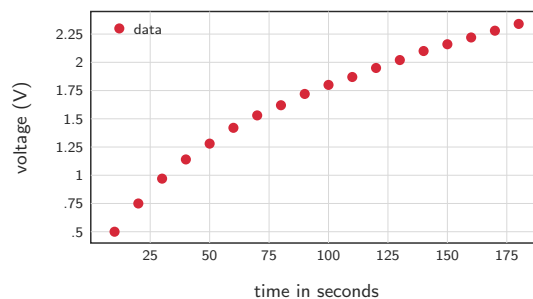
# your data text data that you read in. Or a \r might work, or
# both! If you are having difficult output, try this.
# The next line also makes sure our data line is in string
# format.
writeString=str(arduinoData) #+ " \n"
# Of course it could be that you have a \n and don't need one.
# The next line removes the end of line characters in case you
# have extras.
writeString = writeString.replace("\n", "")
# The next line writes our time plus Arduino value to the file. If
# it doesn't have a new line character, it will supply one (well,
# sometimes. If you don't need this line, comment it out)
dataFile.write(writeString)
# And finally we increment the loop counter so we can go get the
# line of data.
i=i+1 # end Data collection loop

# We are done! Print out a message saying we are done
print("done with data collection, closing the file and the serial port"
      )

# Close the file
dataFile.close()
# Close the serial port
ser.close()
# Note if you code does not finish running, your file will still be
# open
# and your serial port will still be locked to the code. That means
# running the code again won't work. So if this happens, type the last
# two commands in the console window to close the file and the
# serial port.
#-----
#-----

```

The resulting data could be plotted in python. It might look something like this.



I plotted this in python. You might plot it in Excel or LoggerPro or many other plotting tools. Before we fit a line to data. But looking at the data, a linear fit doesn't seem appropriate for this data. So we need to go beyond what we have done so far. To do this we need a curve fit tool that can use whatever equation that might fit the data. Python can do this, and so can LoggerPro. But Excel can't. We will have to be more picky about what tool to use to match our data.

6.2.1 Python Advanced Curve Fitting.

In a previous lab we fit a line to a set of data in python. That wasn't very hard. But looking at the sample data in the previous graph, it is clear that a straight line won't do. The data has a definite curve to it. So what equation do we use for our fit? The only equation that makes sense is the equation we developed from our reasoning about RC circuits.

$$\Delta V_C(t) = \Delta V_{\max} \left(1 - e^{-\frac{t}{\tau}}\right) \quad \text{charging}$$

We can use a different python routine to fit a curvy equation like this. We will use the scipy optimize curve.fit() routine. But we can help the curve.fit() routine a little by adding in an extra part to our equation. Let's code our fit equation like this

$$y = A * (1 - \exp(-B * x)) + C$$

where we have added a constant C . Of course C should be equal to zero or at least should be close to zero because there is no constant term in our theoretical equation. We will need to check for this in our curve fit solution.

[Download here](#)

```
#####
# Program to do a curve fit in python with a user defined
# fit equation and % with data supplied by the user.
# The user types the data into numpy arrays by hand,
# and supplies a function with the fit equation.
# In this example, the function is called RC_Charging.
```

```

#
# Author:   Todd Lines
# Date:    2023-02-20
#####
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Define the Gaussian function
print("__")
print("Find_a_curve_fit_to_a_user_defined_function")

#Here is where you define the function to use in the fit
def RC_Charging(x, A, B, C):
    y = A*(1-np.exp(-B*x)) + C
    return y

#here is where you give the data to fit. Put it into numpy
# arrays. Say our data is voltage vs. time. Put the
# time # values in the xdata array and the voltage
# values in the # ydata array. For the exmple the units
# are seconds and volts
xdata = np.array([10, 20, 30, 40, 50, 60, 70, 80,
                  90, 100, 110, 120, 130, 140, 150,
                  160, 170, 180])
ydata = np.array([0.5, 0.76, 0.97, 1.15, 1.3, 1.42,
                  1.54, 1.64, 1.73, 1.82, 1.9, 1.97,
                  2.05, 2.12, 2.18, 2.24, 2.28, 2.33])

#we also need the ydata uncertainty for error bars.
# Say it is 0.1 (you should calculate what your
# value should be)
ydata_err = 0.1 #V

#Now plot the data so we can see the data points
plt.plot(xdata, ydata, 'o')

#Now perform the curve fit. We can't_just_use_a_linear
# fit. The data is very much not linear. So we
# will use a more robust curve fit routine from scipy.
# The scipy optimize curve_fit() routine needs the
# equation to use for the fit as a function (here
# RC_Charging()) and it needs the fit parameters and
# for the error on the fit parameters we need the
# covariance matrix to be output

```

```

parameters, covariance = curve_fit(RC_Charging,
                                   xdata, ydata)

#Pull out the fit parameters
fit_A = parameters[0]
fit_B = parameters[1]
fit_C = parameters[2]

#Pull out the uncertainty from the diagonal elements
# of the covariance matrix. Remember that the
# diagonal elements are the error squared.
SE = np.sqrt(np.diag(covariance))
SE_A = SE[0]
SE_B = SE[1]
SE_C = SE[2]

#Use the fit parameters to make a set of estimated
# y values from the fit equation. We can pass
# in the whole xdata array and get out all the
# y value estimates at once using our function
# with our fit equation. I called the new y-values
# fit_y.
fit_y = RC_Charging(xdata, fit_A, fit_B, fit_C)

#Plot the data (as dots) and the fit (as a line) to
# see if the equation makes sense as a good fit.
plt.xlabel('Time_in_seconds')
plt.ylabel('Voltage_(V)')
plt.errorbar(xdata, ydata, yerr = ydata_err,
             label = 'data', fmt='ro')
plt.plot(xdata, fit_y, 'b-', label='fit')
plt.legend()
plt.show()

#and print out our fit parameters and their
# uncertainties
print("_")
print('The_value_of_A_is_', fit_A, end = "_")
print('with_standard_error_of_', SE_A)
print('The_value_of_B_is_', fit_B, end = "_")
print('with_standard_error_of_', SE_B)
print('The_value_of_C_is_', fit_C, end = "_")
print('with_standard_error_of_', SE_C)
print("_")

```

```
#sometimes the curve fit routine throws a
# math warning, let the user know that the program
# ended and not to be upset about the warning
print('successful_end_of_program')
print('warning_about_overflow_may_follow')
print("_")
```

We have to match our variables with the ones we have in our theory equation. Let's compare the equations.

$$y = A * (1 - \exp(-B * x)) + C$$

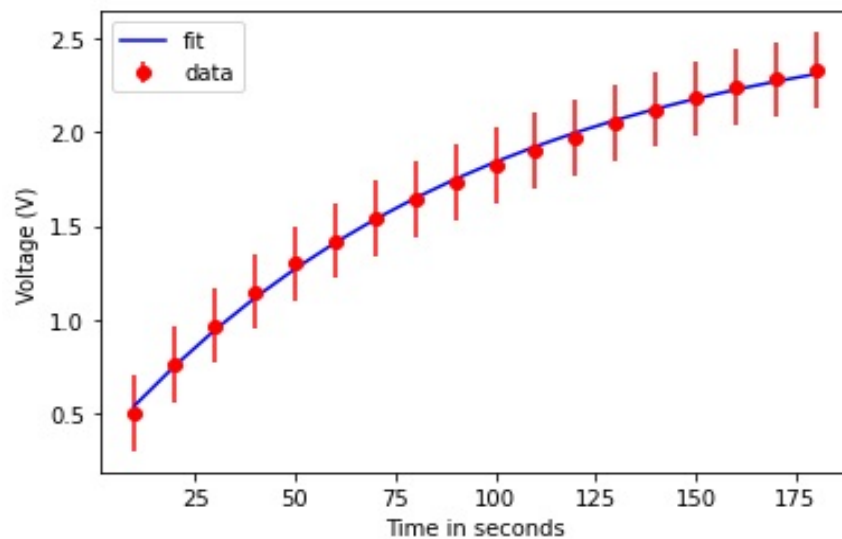
$$\Delta V_C(t) = \Delta V_{\max} \left(1 - e^{-\frac{t}{\tau}}\right) + 0$$

We can see that

Theory	python
$\Delta V_{\max}$	A
0	C
τ	$\frac{1}{B}$

I suppose we could have just used the theory variables like $\Delta V_{\max}$ in python. you might choose to do that.

Our curve fit might look like this



The program output looks like this:

```
Find a curve fit to a user defined function
```

```
The value of A is 2.35419 with standard error of 0.03356.
The value of B is 0.01051 with standard error of 0.00047.
The value of C is 0.30913 with standard error of 0.02340.
```

```
successful end of program, warning about overflow may follow
```

If you look at the code you will notice that there is a big difference between the `linregress()` function and our `curve_fit()` function when it comes to the uncertainties in the coefficients. The `linregress()` function just gave the uncertainties. But the `curve_fit()` function returns a covariance matrix. The diagonal elements of this matrix are our uncertainties on our coefficients – squared! So we had to take a square root to get the uncertainties.

The curve fit looks nice and that is comforting. For my data, it looks like our capacitor model might be correct, an important thing to check is to see if the curve fit goes through the error bars on all of the data. If it does, it is a good sign that the curve fit might represent the data well. Beware! you have to put your own estimate for the uncertainties of your voltage measurements into the code! I overestimated the size of the error bars on purpose to make sure you couldn't miss them in the plot. Yours might be smaller (or bigger!) depending on how you design your instrument.

But now let's go back to our little list of curve fit parameters. We identified

$$\tau = \frac{1}{B}$$

and for my data I have

$$B = 0.01051 \pm 0.00047$$

We need units, and looking at the equation we know τ has units of seconds, so B must have units of inverse seconds.

$$B = (0.01051 \pm 0.00047) \frac{1}{s}$$

so we can find a value for τ . For my data, I have

$$\begin{aligned} \tau_{measured} &= \frac{1}{0.01051 \frac{1}{s}} \\ &= 95.15488s \end{aligned}$$

Note that we will have to calculate the uncertainty in τ . I will leave that for an exercise. But I can compare this $\tau_{measured}$ to the $\tau = RC$ value I started with. If they are within each other's error range, this is a powerful confirmation of our capacitor model.

6.3 Lab Assignment

We have two equations for charge as a function of time for a RC circuit. They are

$$\Delta V_C(t) = \Delta V_{\max} \left(1 - e^{-\frac{t}{\tau}}\right) \quad \text{charging}$$

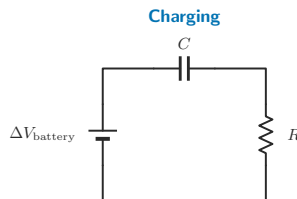
$$I(t) = I_{\max} e^{-\frac{t}{\tau}} \quad \text{charging}$$

where

$$\tau = RC \tag{6.2}$$

is the time constant.

1. Using a capacitor with a capacitance of about $20\mu\text{F}$ and a resistor of about $1\text{M}\Omega$, create a circuit as shown. This will be our system that we will use to test our capacitor model.



You can use one of our power supplies, but be careful to either stay in the 0V to +5V range, or to use a voltage divider to achieve this range at the Arduino input. Follow good lab notebook procedures by recording the model you are testing and your test setup in your lab notebook. Just a note, we will be using directional capacitors. You haven't studied directional capacitors in class. For today's lab, the only difference is that these capacitors only work one direction. This is a little like our diodes. If the circuit doesn't work, try turning your capacitor around.

2. Build your instrument. and write the sketch and Python collection codes. Test every part of the instrument before you start collecting capacitor data. Don't forget to find your uncertainties. If you start your python code but it doesn't finish, remember to close the serial port and the file before trying again. Follow good lab notebook procedures by recording your instrument design in your lab notebook.
3. Now get ready to collect data for a capacitor charge. Work with a lab partner from your group to achieve the data collection. Compare your data among your group to make sure things went well. Follow good lab notebook procedures by recording your data or giving a location of the stored data in your lab notebook.

4. Take the data from your file and graph it. Graphing is easy in python, but you could use LoggerPro if you have this on your computer. You should include this graph in your lab notebook (but might also include the curve fit described in next item on the same graph).
5. Perform the curve fit. As in the last lab, having the proper curve fit the data is a validation of our model! So if the theoretical curve fits the data, it makes sense that something about the model might be right. Note that we have an equation from our theory, equation (6.1). If our data fits a different curve, that could be evidence that our theory is wrong! We hope the data fits this curve and no other. We know python can do this and so can LoggerPro. Python might be a good choice. Once you have the curve fit and the fit parameters (and their uncertainties) make a graph of the data and the fit. Error bars would be a good idea (but don't use my uncertainty values, calculate your own!). Include the graph of the curve fit and the data in your lab notebook as well as the fit equation and fit parameters (don't forget their uncertainties).
6. Find the time constant and compare to your predicted value. If these compare within their uncertainties, we have a further validation of the model. Record the time constants and their uncertainties in your lab notebook.
7. Draw a conclusion, is our capacitor model good?

